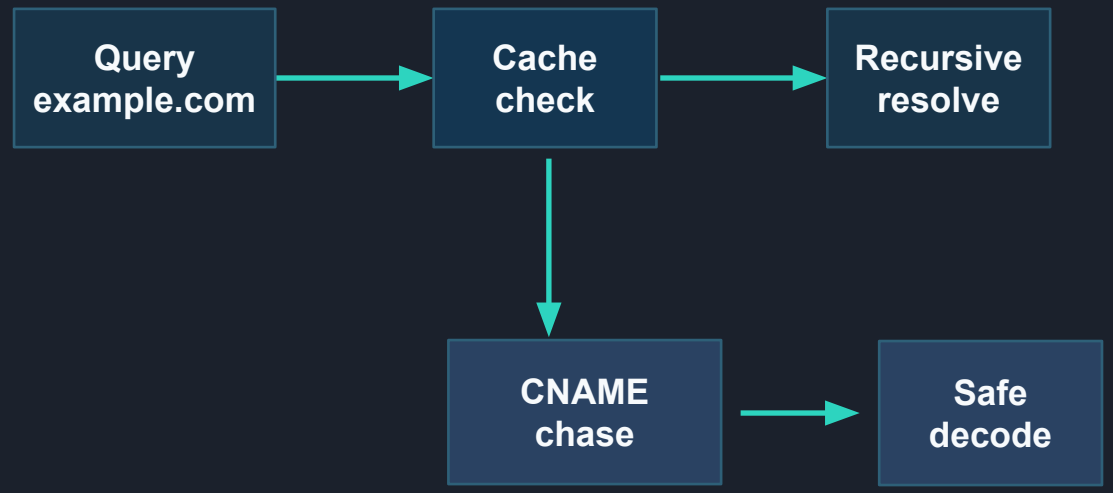
DNS Resolver Phase 3

CNAME chasing, and loop-safe decoding for a CS-158 recursive DNS resolver.

part 1: CNAME

part 2: Compression Loop Detection



Authors: Braden Diaz De Leon and Franklin Du

Part 1: CNAME Handling

Part 1: CNAME chasing for real-world aliases

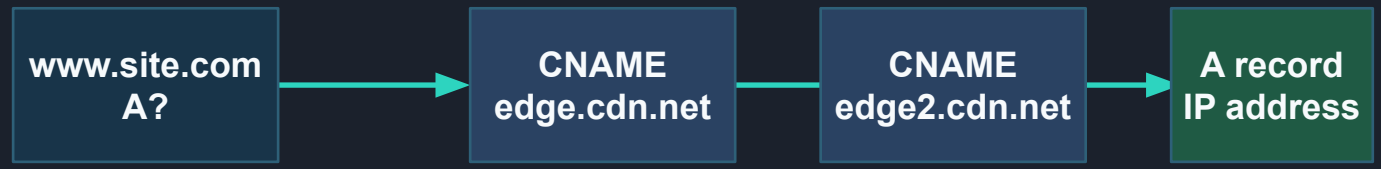
PART 1

Many real websites answer an A-record query with an alias first.

- Baseline only matched A records. A CNAME-only answer meant "something went wrong" or an infinite referral loop
- A record wins when a server bundles both, no extra round trip
- Otherwise restart at the canonical name from the root, the alias usually points into a different zone
- Alias cached under its own key and TTL repeat lookups resolve with zero packets
- Every alias followed is recorded. Revisiting one is an error not a hang

Why this matters

CDNs, load balancers, GitHub Pages, and platform managed domains use aliases. Chasing CNAMEs makes the resolver useful on real hostnames.




```
resolve(name, qtype):
    server, visited = ROOT, { }

    while True:
        if (name, qtype) in cache:
            return cache[name, qtype]

        if qtype != CNAME and (name, CNAME) in cache:
            target = cache[name, CNAME]
        else:
            reply = query(server, name, qtype)

            if addr := get_answer(reply):
                cache[name, qtype] = addr with ttl
                return addr

            if not (target := get_cname(reply)):
                server = next_nameserver(reply)
                continue

            cache[name, CNAME] = target with the CNAME's own ttl

        # both paths converge here
        if name in visited: raise CNAMELoopError(name)
        visited.add(name)
        name, server = target, ROOT
```

RFC basis: why the fixes match DNS standards

CNAME RFCs

RFC 1035 §3.3.1: CNAME data is a domain name so decode it with pointer support.

RFC 1034 §4.3.2: when QTYPE is not CNAME restart at the canonical name.

RFC 1034 §3.6.2: follow CNAME chains and signal loops as errors.

RFC 2181 §10.1: CNAME only answers are valid aliases, not failures.

The resolver behavior matches DNS standards and handles real-world failure modes safely.

Part 1: Testing CNAME Handling

Test Cases

- Test 1: CNAME Chasing
  - Test 1: multi-hop CNAME chain

```
Cache Miss.
Querying 198.51.44.6 for www.prd.map.nytimes.xovr.nyt.net
CNAME: www.prd.map.nytimes.xovr.nyt.net -> nytimes.map.fastly.net (restarting query at canonical na
me)

Checking Caches...

Cache Miss.
Querying 198.41.0.4 for nytimes.map.fastly.net
Checking Caches...

Cache Miss.
Querying 192.55.83.30 for nytimes.map.fastly.net
Checking Caches...

Cache Miss.
Querying 23.235.32.32 for nytimes.map.fastly.net
Fetched IP: 151.101.201.164
dig IP's: {'151.101.201.164'}
Correct IP!
```

```
Querying 198.41.0.4 for ns-1029.awsdns-00.org
Querying 199.249.112.1 for ns-1029.awsdns-00.org
Querying 205.251.192.128 for ns-1029.awsdns-00.org
Querying 205.251.196.5 for www.reddit.com
Querying 198.41.0.4 for ns-1029.awsdns-00.org
Querying 199.249.112.1 for ns-1029.awsdns-00.org
Querying 205.251.192.128 for ns-1029.awsdns-00.org
Querying 205.251.196.5 for www.reddit.com
Querying 198.41.0.4 for ns-1029.awsdns-00.org
Querying 199.249.112.1 for ns-1029.awsdns-00.org
Baseline part_3.resolve: HUNG (infinite referral loop)
Checking Caches...
```

- Test 2: CNAMEs interacting with the cache
  - Test 0: caches records
  - Test 1: checks and uses the cached records

```
Cache Miss.
Querying 208.80.154.238 for dyna.wikimedia.org
Fetched IP: 198.35.26.224
dig IP's: {'198.35.26.224'}
Correct IP!

[(1785624731.51359, 'dyna.wikimedia.org', 1), (1785710951.361003, 'en.wikipedia.org', 5)]
{('en.wikipedia.org', 5): 'dyna.wikimedia.org', ('dyna.wikimedia.org', 1): '198.35.26.224'}
```

```
Test 1:
Checking Caches...

Found CNAME: en.wikipedia.org -> dyna.wikimedia.org

Checking Caches...

Found Record: 198.35.26.224

Fetched IP: 198.35.26.224
dig IP's: {'198.35.26.224'}
Correct IP!

[(1785624731.51359, 'dyna.wikimedia.org', 1), (1785710951.361003, 'en.wikipedia.org', 5)]
{('en.wikipedia.org', 5): 'dyna.wikimedia.org', ('dyna.wikimedia.org', 1): '198.35.26.224'}
```

- Test 3: CNAME Loop Detection
  - Test 0: a.example -> b.example -> a.example must be signalled as an error

```
Test 0: a.example <-> b.example (CNAME cycle)
Checking Caches...

No Expired Records
Cache Miss.
Querying 198.41.0.4 for a.example
CNAME: a.example -> b.example (restarting query at canonical name)

Checking Caches...

Cache Miss.
Querying 198.41.0.4 for b.example
CNAME: b.example -> a.example (restarting query at canonical name)

Checking Caches...

Found CNAME: a.example -> b.example

Loop detected: CNAME loop detected: a.example already visited in this chain
Correct!
```

Key Takeaway: The resolver can now handle hostname aliases with CNAME chasing and loop detection

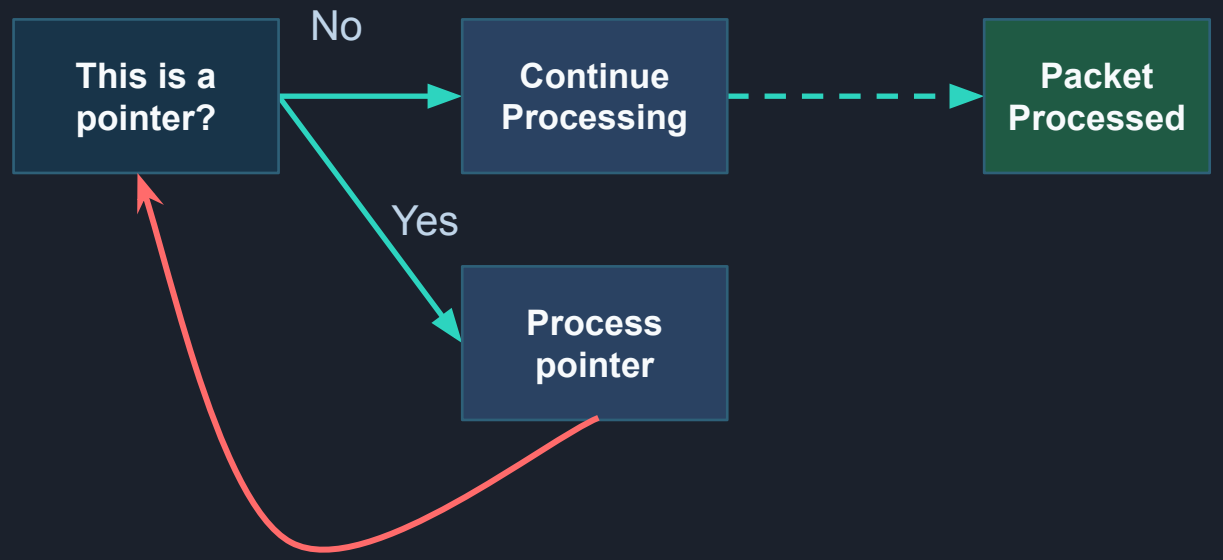
Part 2: Compression Loop Detection

Part 2: Compression infinite loop detection

PART 2

Goal: Avoid being stuck in infinite loops while decoding compressed names.

- added a loop counter for `decode_compressed_name` in `part_2.py`
- Every pointer adds a counter to the loop
  - Max limit of compression pointers is 135



Why this matters

If there is no way to detect an infinite loop, then the resolver would be stuck processing the pointers and eventually hit an error due to infinite recursions. This is a type of attack that can be used against the resolver: a DoS attack.

```
decode_compressed_name:
    counter +=1
    if counter exceeds maxCompressionLoops:
            raise exception
    process pointer
```

RFC basis: why the fixes match DNS standards

Loop Detection RFCs

RFC 9267: Infinite loop detection not mandatory but advised.

RFC 1035 §2.3.4: There is a limit to 255 octets for a domain name.

RFC 1035 §4.1.4: Compression pointers consists of two octets.

> These two sections in RFC 1035 imply the limit of pointers in an input: 255/2 = 127.5.
> Rounded up and given some buffer, a reasonable limit is 135 compression pointers.

The resolver behavior matches and enforces DNS standards while increases security and error handling capabilities to match the needs of the real-world.

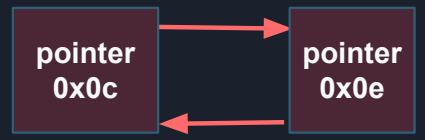
Part 2: testing

```
Test: 0

Caught Exception Compression Loop Detected

Number of Hops: 135
```

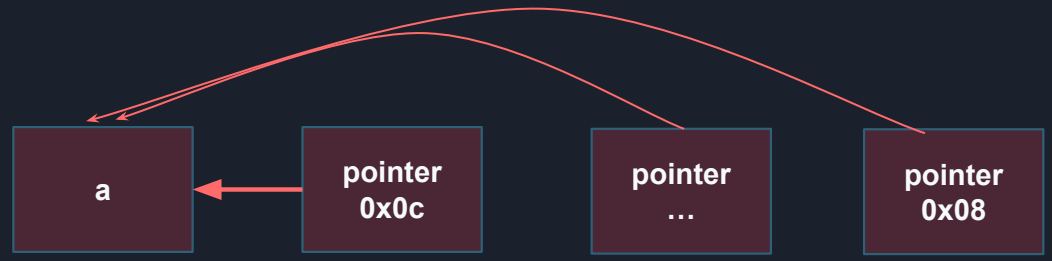
```
Test: 1

Caught Exception Compression Loop Detected

Number of Hops: 135
```

WHAT IS TESTED

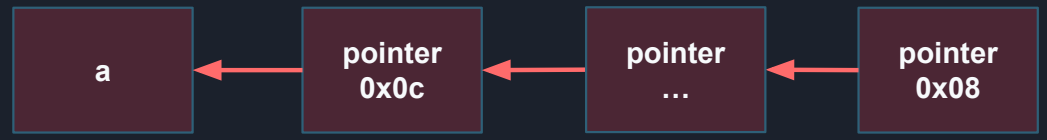
- Test 0:
  Single-hop infinite loops.
- Test 1:
  Double-hop infinite loops.
- Theoretical max number of legal pointers.
  - Test 2:
    Pointing at the same label
  - Test 3:
    Chained together




```
Caught Exception maximum recursion depth exceeded
Phase 2 Tests Complete!
```




```
Test: 2

No Loop Detected

Number of Hops: 1
```




```
Test: 3

No Loop Detected

Number of Hops: 126
```

Key takeaway

The resolver is more secure from attacks, and only reads and uses legal compressed inputs.

Thank You